

1. CONOCIMIENTOS SQL

1.1) Describe el funcionamiento general de la sentencia JOIN.

La sentencia JOIN es utilizada para unir dos o más entidades por medio de sus llaves primarias y foráneas, su funcionamiento consiste en cruzar datos de ambas entidades fila por fila basándose en coincidencias según la condición establecida después del ON.

1.2) ¿Cuáles son los tipos de JOIN y cuál es el funcionamiento de los mismos?

Existen varios tipos:

INNER JOIN: Este tipo de sentencia devuelve solo los registros coincidentes entre ambas entidades. Ejemplo si un registro de la entidad A no tiene relación con el de la entidad B no se muestra.

LEFT JOIN: Esta sentencia LEFT JOIN o LEFT OUTER JOIN devuelve todos los registros de la entidad de la izquierda (ENTIDAD A) y las coincidencias con la entidad de la derecha (ENTIDAD B), si no hay coincidencias, los campos de la entidad B se rellenan con NULL.

RIGHT JOIN: Esta sentencia RIGHT JOIN o RIGHT OUTER JOIN funciona al revés del LEFT JOIN devuelve todos los registros de la entidad de la derecha (ENTIDAD B) y las coincidencias de la entidad de la izquierda (ENTIDAD A). Si no hay coincidencia el lado izquierdo muestra NULL.

FULL JOIN: Devuelve todos los datos de ambas tablas cuando hay coincidencias en una de ellas, si no hay coincidencia rellena con NULL los lados correspondientes.

1.3) ¿Cuál es el funcionamiento general de los TRIGGER y qué propósito tienen?

Un TRIGGER o disparador es un código SQL que está asociado a una entidad que se ejecuta automáticamente cuando ocurre un evento específico de manipulación de datos: INSERT, UPDATE, DELETE. Puede ejecutarse antes (BEFORE) o después (AFTER) del evento.

Su propósito es mantener una auditoría de cambios, es decir como mantener un historial de cambios, esto automatiza tareas, validadatos complejos antes de guardarlos o para mantener la integridad referencial del sistema.

1.4) ¿Qué es y para qué sirve un STORED PROCEDURE?

Un store procedure es un bloque de código SQL que contiene instrucciones precompiladas que se almacenan en el servidor de la BD y se pueden llamar CALL "mi_storeProcedure".

Este puede declarar parámetros de entrada y devolver parámetros de salida.

Sirve para centralizar la lógica de negocio pesada en la BD, mejora el rendimiento y aumenta la seguridad evitando inyecciones SQL.

Considerando que las siguientes tablas:

productos	ventas
PK idProducto INT(6)	PK idVenta INT(6)
nombre VARCHAR(40)	FK1 idProducto INT(6)
precio DECIMAL(16,2)	cantidad INT(6)

Tienen los siguientes datos insertados:

idProducto	nombre	precio
1	LAPTOP	3000.00
2	PC	4000.00
3	MOUSE	100.00
4	TECLADO	150.00
5	MONITOR	2000.00
6	MICROFONO	350.00
7	AUDIFONOS	450.00

idVenta	idProducto	cantidad
1	5	8
2	1	15
3	6	13
4	6	4
5	2	3
6	5	1
7	4	5
8	2	5
9	6	2
10	1	8

1.5) Traer todos los productos que tengan una venta.

```
SELECT DISTINCT p.idProducto, p.nombre, p.precio
FROM productos p
INNER JOIN ventas v
ON p.idProducto = v.idProducto;
```

1.6) Traer todos los productos que tengan ventas y la cantidad total de productos vendidos.

```
SELECT p.idProducto, p.nombre, SUM(v.cantidad) AS cantidad_total_vendida
FROM productos p
INNER JOIN ventas v ON p.idProducto = v.idProducto
GROUP BY p.idProducto, p.nombre;
```

1.7) Traer todos los productos (independientemente de si tienen ventas o no) y la suma total (\$) vendida por producto.

Aquí se tendría que usar el LEFT JOIN para no ignorar los productos sin ventas:

```
SELECT p.idProducto, p.nombre, COALESCE(SUM(v.cantidad * p.precio), 0.00) AS  
total_dinero_vendido  
FROM productos p  
LEFT JOIN ventas v ON p.idProducto = v.idProducto  
GROUP BY p.idProducto, p.nombre;
```

2. EJERCICIO PRÁCTICO: BD

2.1) Crea un diagrama relacional de BD para el escenario descrito anteriormente.

